

CUDA — Memory Model



Raj Prasanna Ponnuraj · [Follow](#)

Published in Analytics Vidhya

6 min read · Oct 9, 2020



Listen



Share

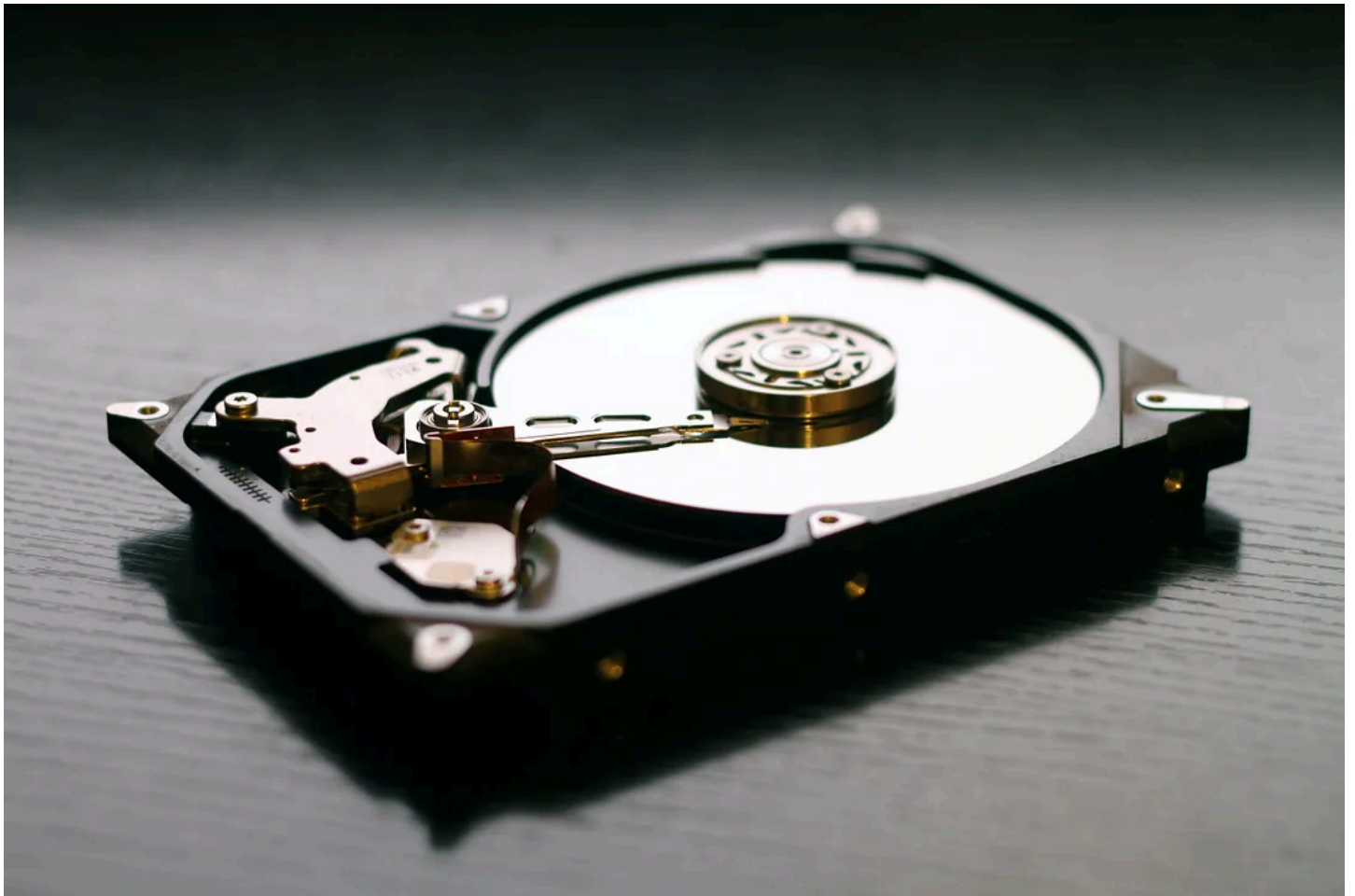


Photo by [Picsaf Com](#) from [Pexels](#)

This post details the CUDA memory model and is the fourth part in the [CUDA series](#).

Part 1 — [Heterogenous Computing](#)

Part 2 — [CUDA Kernels and their Launch Parameters](#)

Part 3 — [GPU Device Architecture](#)

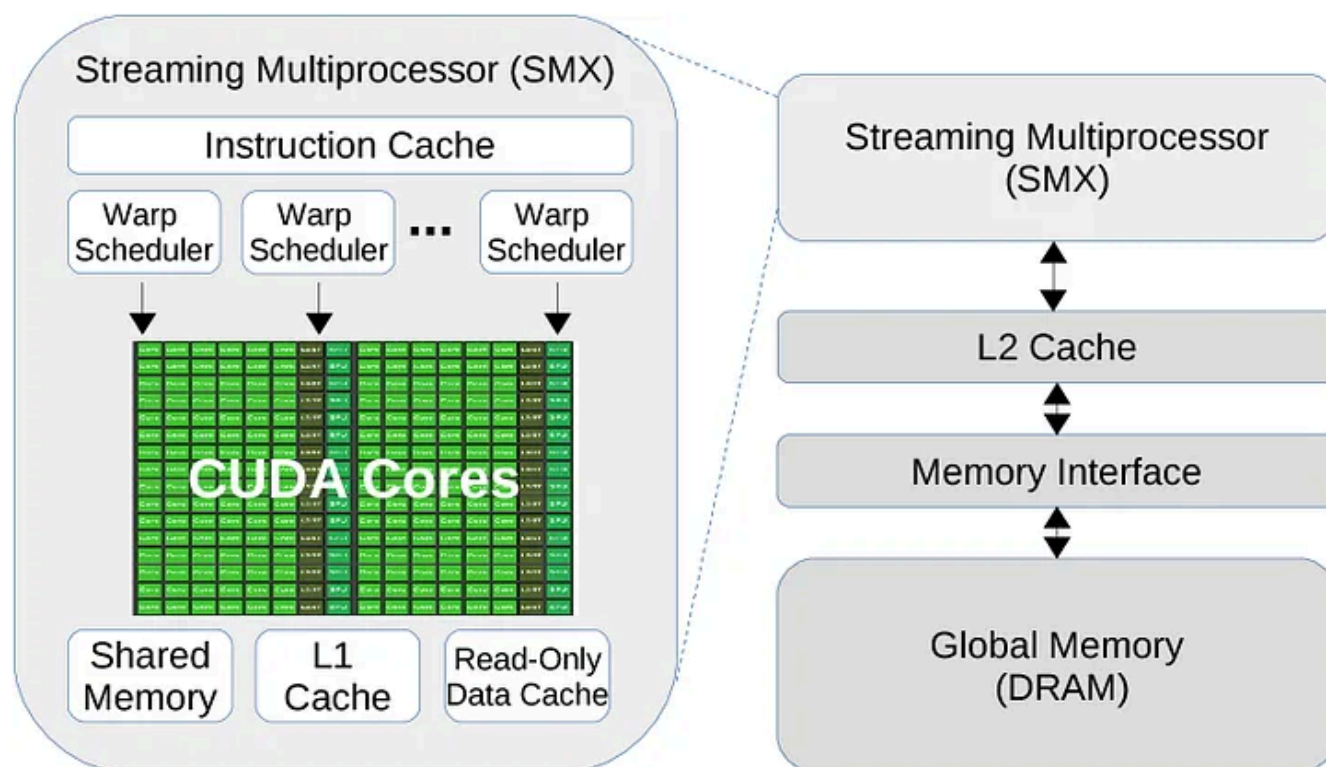
Memory Hierarchy

During the execution of a computer application, more often the instructions have the tendency to access the same set of memory locations repeatedly over a short period of time. This phenomenon is called principle of locality. There are two types of locality — temporal locality, spatial locality.

Temporal locality — the tendency to access the same memory location repeatedly with in a relatively short period of time.

Spatial locality — the tendency to access the memory locations with in a relatively close proximity to the currently accessed location.

Due to the existence of this principle, any computer architecture will have a hierarchy of memory, thereby optimizing the execution of the instructions. As the distance of the memory increases from the processor, the data access from that memory take more clock cycles to process.



GPU Memory Hierarchy

In case of an NVIDIA GPU, the shared memory, the L1 cache and the Constant memory cache are within the streaming multiprocessor block. Hence they are faster than the L2 cache, and GPU RAM.

GPU Execution model

As discussed in [Part 1](#) of this series, GPU is a co-processor. GPU kernel launch, and data initialization and transfer happens from the CPU. Let's take an example to discuss further.

```
__global__ void array_sum (int *d_a, int *d_b, int *d_c, int size)
{
    int gid = blockDim.x * blockIdx.x + threadIdx.x;
    if (gid < size)
        c[gid] = a[gid] + b[gid];
}
```



In order to process the above kernel in GPU, arrays a, b and c should be initialized in the CPU and transferred to the GPU. So, the main function which executes in the CPU should be something as below.

```
int main()
{
    int size = 1 << 20;
    int block_size = 128;
    int NO_BYTES = sizeof(int) * size;

    //Host memory allocation
    int *h_a, *h_b, *h_c;
    h_a = (int *)malloc(NO_BYTES);
    h_b = (int *)malloc(NO_BYTES);
    h_c = (int *)malloc(NO_BYTES);

    //Host memory initialization
    for(int i = 0; i < size; i++)
    {
        h_a[i] = 10;
        h_b[i] = 20;
    }
    memset(h_c, 0, NO_BYTES);

    //Device memory initialization
    int *d_a, *d_b, *d_c;
    cudaMalloc((int **)&d_a, NO_BYTES);
    cudaMalloc((int **)&d_b, NO_BYTES);
    cudaMalloc((int **)&d_c, NO_BYTES);

    //Host to device input data transfer
    cudaMemcpy(d_a, h_a, NO_BYTES, cudaMemcpyHostToDevice);
    cudaMemcpy(d_b, h_b, NO_BYTES, cudaMemcpyHostToDevice);

    //Kernel launch
    dim3 block(block_size);
    dim3 grid(size/block.x);

    array_sum <<< grid, block >>> (d_a, d_b, d_c, size);
    cudaDeviceSynchronize();

    //Device to host output data transfer
    cudaMemcpy(d_b, h_b, NO_BYTES, cudaMemcpyDeviceToHost);

    free(h_a);
    free(h_a);
    free(h_a);

    cudaFree(d_a);
    cudaFree(d_b);
    cudaFree(d_c);
}
```

malloc — to allocate memory in the host memory

cudaMalloc — to allocate memory in the device memory

cudaMemcpy — to copy the data from host to device or from device to host. The cudaMemcpyHostToDevice and cudaMemcpyDeviceToHost flags are used to dictate the direction of data transfer.

free — to recover the allocated host memory

cudaFree — to recover the allocated device memory

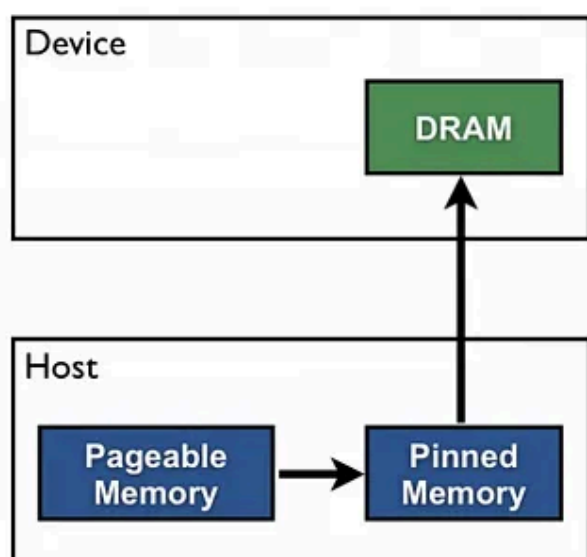
Memory Allocation Types

There are four types of memory allocation in CUDA.

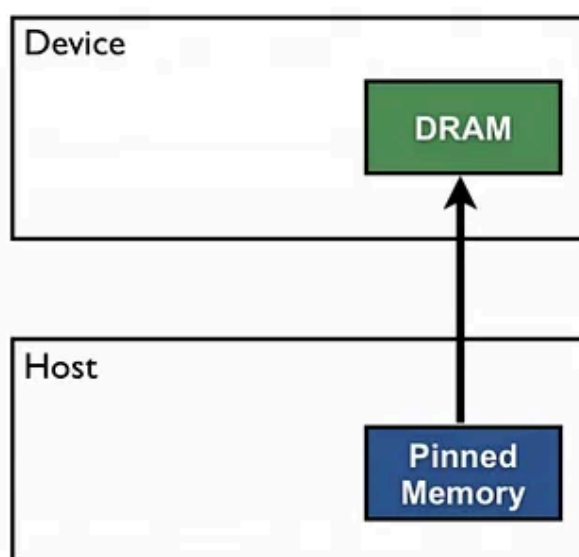
1. Pageable memory
2. Pinned memory
3. Mapped memory
4. Unified memory

Pageable memory

Pageable Data Transfer



Pinned Data Transfer



The memory allocated in host is by default pageable memory. The data at this memory location is usable by the host. To transfer this data to the device, the CUDA run time copies this memory to a temporary pinned memory and then transfers to the device memory. Hence, there are two memory transfers. Therefore, this type of memory allocation and transfer is slow.

Host allocation syntax:

```
int *h_a, *h_b, *h_c;
h_a = (int *)malloc(NO_BYTES);
h_b = (int *)malloc(NO_BYTES);
h_c = (int *)malloc(NO_BYTES);
```

```

for (int i = 0; i < size; i++)
{
    h_a[i] = 10;
    h_b[i] = 20;
}

memset(h_c, 0, NO_BYTES);

```

Device allocation syntax:

```

int *d_a, *d_b, *d_c;
cudaMalloc((int **)&d_a, NO_BYTES);
cudaMalloc((int **)&d_b, NO_BYTES);
cudaMalloc((int **)&d_c, NO_BYTES);

cudaMemcpy(d_a, h_a, NO_BYTES, cudaMemcpyHostToDevice);
cudaMemcpy(d_b, h_b, NO_BYTES, cudaMemcpyHostToDevice);

```

Pinned Memory

The data can be initialized directly in the host pinned memory. By doing so we can avoid two data transfers as in pageable memory. This will make the process faster but at the cost of host performance. As the data is initialized in the pinned memory, the memory availability for host processing is reduced.

Host allocation Syntax:

```

int *h_a1, *h_b1, *h_c1;
cudaMallocHost((int **)&h_a1, NO_BYTES);
cudaMallocHost((int **)&h_b1, NO_BYTES);
cudaMallocHost((int **)&h_c1, NO_BYTES);

for (int i = 0; i < size; i++)
{
    h_a1[i] = 10;
    h_b1[i] = 20;
}

memset(h_c1, 0, NO_BYTES);

```

Device allocation syntax:

```

int *d_a1, *d_b1, *d_c1;
cudaMalloc((int **)&d_a1, NO_BYTES);
cudaMalloc((int **)&d_b1, NO_BYTES);
cudaMalloc((int **)&d_c1, NO_BYTES);

cudaMemcpy(d_a1, h_a1, NO_BYTES, cudaMemcpyHostToDevice);
cudaMemcpy(d_b1, h_b1, NO_BYTES, cudaMemcpyHostToDevice);

```

Mapped memory (zero-copy memory)

Zero copy memory is pinned memory that is mapped into the device address space. Both host and device have direct access to this memory.

Pros:

1. Can leverage host memory when there is insufficient device memory.
2. Can avoid explicit data transfers between host and device.
3. Improves PCIe transfer rates

Cons:

1. As it is mapped into the device address space, the data will not be copied into the device memory. Transfer will happen during execution which will increase the processing time considerably.

Host allocation syntax:

```
int *h_a2, *h_b2, *h_c2;
cudaHostAlloc((int **)&h_a2, NO_BYTES, cudaHostAllocMapped);
cudaHostAlloc((int **)&h_b2, NO_BYTES, cudaHostAllocMapped);
cudaHostAlloc((int **)&h_c2, NO_BYTES, cudaHostAllocMapped);

for (int i = 0; i < size; i++)
{
    h_a2[i] = 10;
    h_b2[i] = 20;
}

memset(h_c2, 0, NO_BYTES);
```

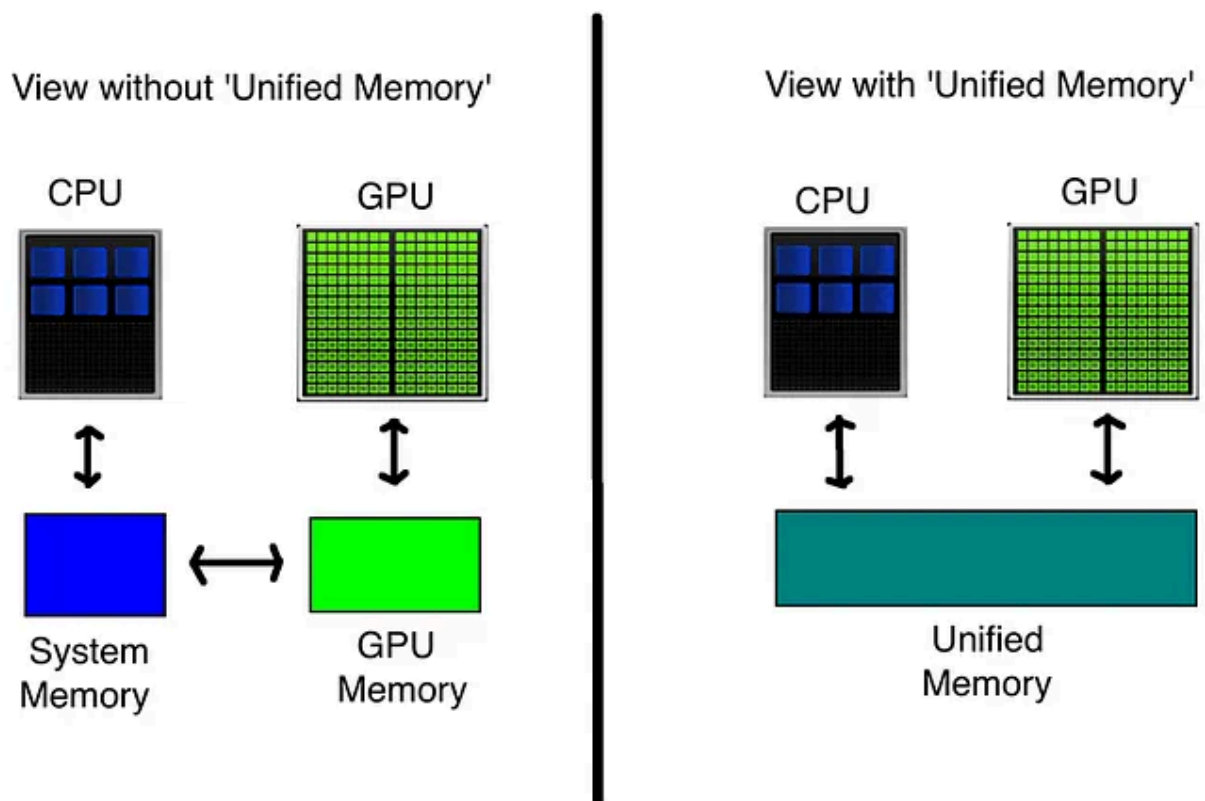
Device allocation syntax:

```
int *d_a2, *d_b2, *d_c2;
//
cudaHostGetDevicePointer((int **)&d_a2, (int *)h_a2, 0);
cudaHostGetDevicePointer((int **)&d_b2, (int *)h_b2, 0);
cudaHostGetDevicePointer((int **)&d_c2, (int *)h_c2, 0);
```

Here we are getting just the device pointer using *cudaHostGetDevicePointer* function and not allocating a new memory for device.

Unified memory

Programmers' View



This creates a pool of managed memory where each allocation from this memory pool is accessible on both the host and the device with the same address or pointer. The underlying system migrates data to host and device.

Pros — No explicit allocation and recovery of memory for device needed. This reduces programming complexity.

Cons — Adds additional instructions under the hood for memory management.

Syntax:

Since this type is unified one, we have only 1 initialization.

```
// int *a, *b, *c;
//
// cudaMallocManaged((int **)&a, NO_BYTES);
// cudaMallocManaged((int **)&b, NO_BYTES);
// cudaMallocManaged((int **)&c, NO_BYTES);
//
// for (int i = 0; i < size; i++)
// {
//   a[i] = 10;
//   b[i] = 20;
```

```
// }  
//  
// memset(c, 0, NO_BYTES);
```

Comparisons

Let's compare and contrast the above four methods in terms of memory transfer time and kernel execution time.

For this comparison, I've take the same *array_sum* kernel with array size as $1 < 20$.

Memory Type	Memory Trasfer time in ms	Kernel Execution Time in ms	Total Time in ms
Pageable	19.43	0.44	19.87
Pinned	4.26	0.44	4.70
Mapped	0.00	3.72	3.72
Unified	1.96	16.13	18.09

The above numbers are obtained by profiling the compiled CUDA code with NVIDIA NSIGHT Systems profiler.

Observations

1. Compared to pageable memory, pinned memory has only 1 memory transfer. Hence memory transfer time is less for pinned memory than pageable memory.
2. In mapped memory, the address is mapped to the device address space. Hence there is no explicit memory transfer. So the transfer time is 0. But since it is similar to pinned memory operation under the hood, total time is more or less similar for both.
3. In unified memory, the data resides in the managed pool and transferred as and when required to the device under the hood. Hence the memory transfer time is less but kernel execution time is more. This type is comparable to the pageable memory but the difference between the two is implicit memory transfers in unified memory.

In the next post, I'll talk about the memory model inside the streaming multiprocessor. Thanks.

[Nvidia](#)[Parallel Computing](#)[Cuda](#)[Computer Architecture](#)[Machine Learning](#)



Follow

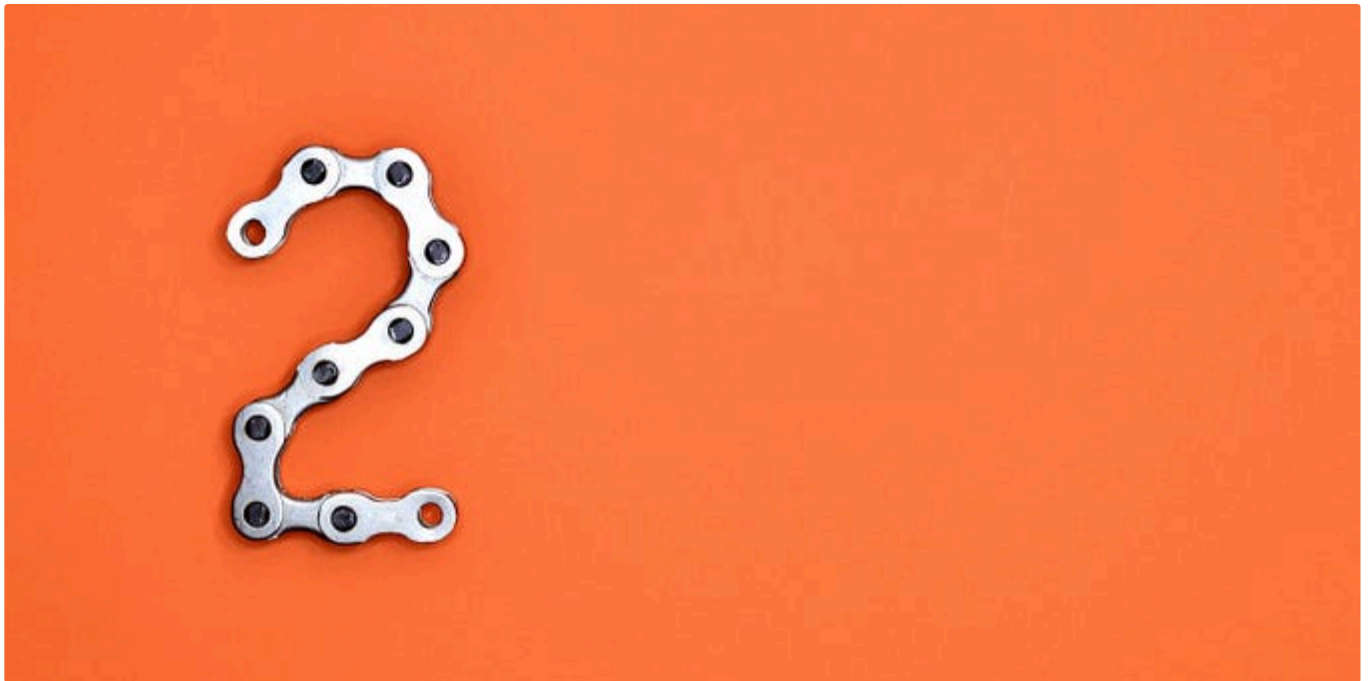


Written by Raj Prasanna Ponnuraj

47 Followers · Writer for Analytics Vidhya

Deep Learning Engineer

More from Raj Prasanna Ponnuraj and Analytics Vidhya



 Raj Prasanna Ponnuraj in Analytics Vidhya

CUDA—Compute Unified Device Architecture—Part 2

This article is a sequel to this article. In this article let's focus on the device launch parameters, their boundary values and the...

Sep 19, 2020  66  2





 Kia Eisinga in Analytics Vidhya

How to create a Python library

Ever wanted to create a Python library, albeit for your team at work or for some open source project online? In this blog you will learn...

Jan 27, 2020  2.7K  28



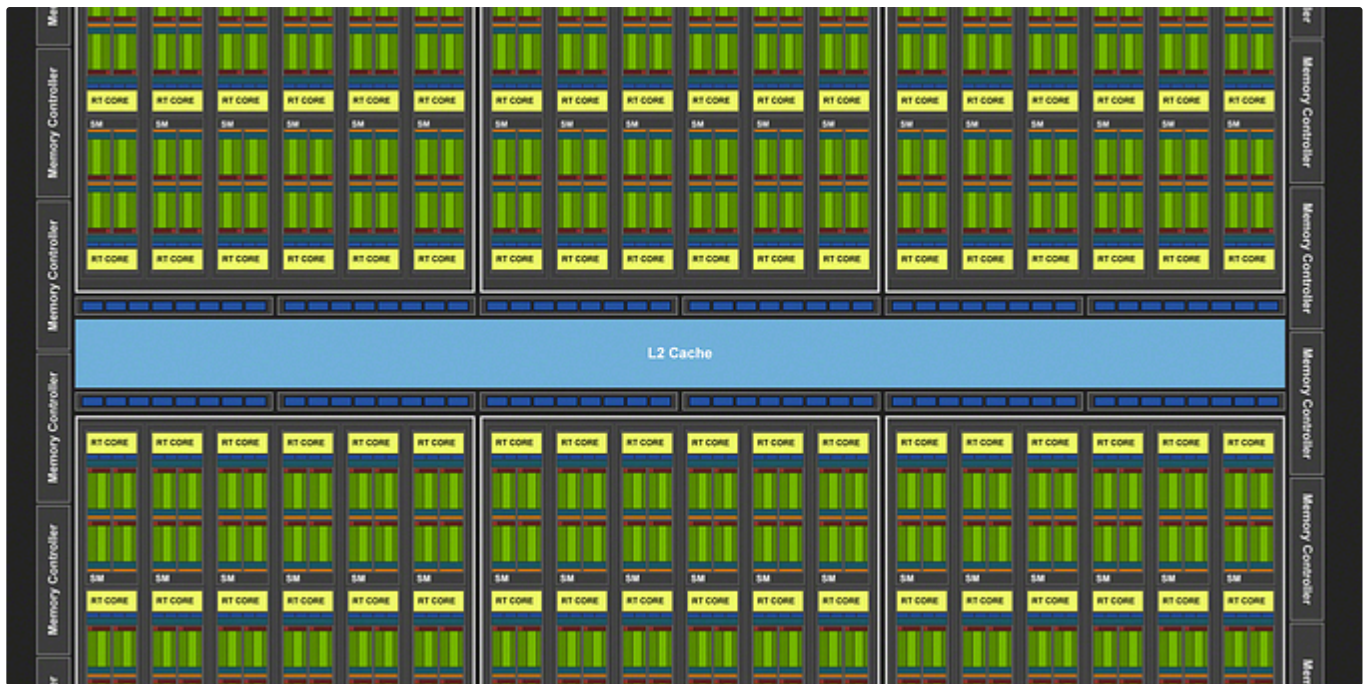
 Harikrishnan N B in Analytics Vidhya

Confusion Matrix, Accuracy, Precision, Recall, F1 Score

Binary Classification Metric

Dec 10, 2019  1.1K  6





 Raj Prasanna Ponnuraj in Analytics Vidhya

CUDA — Compute Unified Device Architecture — Part 3

This post is part 3 in the sequel. The link to previous posts are Part 1 and Part 2. In this post we shall talk about the basic...

Sep 25, 2020  140



See all from Raj Prasanna Ponnuraj

See all from Analytics Vidhya

Recommended from Medium

PyTorch

 Yunusemre

Pytorch & C++ #1

Pytorch & C++ #1

★ Mar 1 🖱 1



AMAZON.COM

Software Development Engineer

SEATTLE, WA

Mar. 2020 – May 2021

- Developed Amazon checkout and payment services to handle traffic of 10 Million daily global transactions
- Integrated Iframes for credit cards and bank accounts to secure 80% of all consumer traffic and prevent CSRF, cross-site scripting, and cookie-jacking
- Led Your Transactions implementation for JavaScript front-end framework to showcase consumer transactions and reduce call center costs by \$25 Million
- Recovered Saudi Arabia checkout failure impacting 4000+ customers due to incorrect GET form redirection

Projects

NinjaPrep.io (React)

- Platform to offer coding problem practice with built in code editor and written + video solutions in React
- Utilized Nginx to reverse proxy IP address on Digital Ocean hosts
- Developed using Styled-Components for 95% CSS styling to ensure proper CSS scoping
- Implemented Docker with Seccomp to safely run user submitted code with < 2.2s runtime

HeatMap (JavaScript)

- Visualized Google Takeout location data of location history using Google Maps API and Google Maps heatmap code with React
- Included local file system storage to reliably handle 5mb of location history data
- Implemented Express to include routing between pages and jQuery to parse Google Map and implement heatmap overlay

 Alexander Nguyen in Level Up Coding

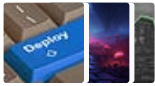
The resume that got a software engineer a \$300,000 job at Google.

1-page. Well-formatted.

★ Jun 1 🖱 19.3K 💬 335



Lists



Predictive Modeling w/ Python

20 stories · 1474 saves



Practical Guides to Machine Learning

10 stories · 1798 saves



Natural Language Processing

1666 stories · 1243 saves



The New Chatbots: ChatGPT, Bard, and Beyond

12 stories · 450 saves



 Canyon Lake Robotics

Tuning the ROS2 Nav2 Stack

If you have a wheeled robot in ROS2, you are almost surely using the Nav2 stack. It provides a ton of great features such as mapping...

★ May 31



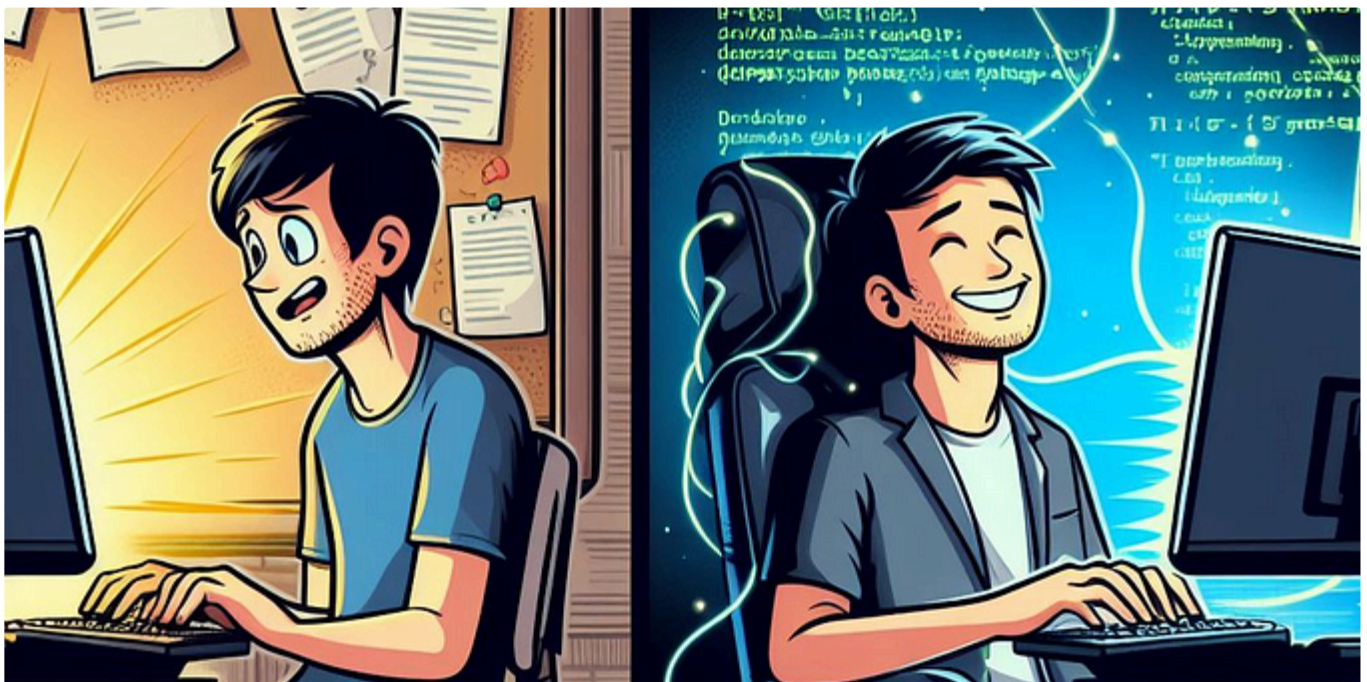


 Muhammad Burhan in Virtual Force Inc.

A step-by-step guide to install NVIDIA Drivers and Cuda Toolkit

A complete guide to setup GPU for ML development.

Mar 14  10



 Abhay Parashar in The Pythoneers

17 Mindblowing Python Automation Scripts I Use Everyday

Scripts That Increased My Productivity and Performance

★ 4d ago  7K  66



```
*__, a, b, *_ = [1, 2, 3, 4, 5, 6]  
print(__, _)
```

What does this print?

- A) Syntax error
- B) [1] [4, 5, 6]
- C) [1, 2] [5, 6]
- D) [1, 2, 3] [6]
- E) <generator object <genexpr> at 0x1003847c0>

 Liu Zuo Lin

You're Decent At Python If You Can Answer These 7 Questions Correctly

No cheating pls!!

★ Mar 6 🖱 7.3K 💬 31



See more recommendations